Moveit Inverse Kinematics Design

Moveit Inverse Kinematics Design
1. Usage Environment
2. Driving the Real Machine
2.1 Starting the Real Device
3. Starting MoveIt2
3.1 Inverse Kinematics Design
Launch Command
3.2. Source Code Analysis

1. Usage Environment

ROS2: Humble

Enter the commands in the tutorial into the accompanying virtual machine to run the function.

2. Driving the Real Machine

Driving the real machine is achieved by subscribing to the Moveit2 `/joint_states` topic, converting the joint state information of the robotic arm into control data for the actual robotic arm.

Note: Since the real robotic arm does not have obstacle avoidance capabilities, some poses may encounter obstacles; therefore, the planned robotic arm movements should be as reasonable as possible and avoid locations with obstacles.

(It is recommended to use preset poses for demonstrating the driving of the real machine.)

The motherboard and virtual machine need to be configured for multi-machine communication. The robotic arm and virtual machine need to be on the same network, and the virtual machine's `ros_domain_id` must be consistent with the `ros_domain_id` in Docker. (Both devices are pre-configured with the same `ros_domain_id` at the factory; only the virtual machine and the robotic arm need to be on the same network.)

2.1 Starting the Real Device

```
# Start this command in Docker,
ros2 run dofbot_ctrl SimulateToArm
```

3. Starting MoveIt2

```
# Start this command in the virtual machine,
ros2 launch dofbot_moveit demo.launch.py
```

```
yahboom@yahboom-VM:~$ ros2 launch dofbot_moveit demo.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-03-13-14-31-56-635718-yahboom-VM-9623
[INFO] [launch]: Default logging verbosity is set to INFO
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
[INFO] [robot_state_publisher-1]: process started with pid [9625]
[INFO] [move_group-2]: process started with pid [9627]
[INFO] [rviz2-3]: process started with pid [9629]
[INFO] [ros2_control_node-4]: process started with pid [9631]
[INFO] [spawner-5]: process started with pid [9633]
[INFO] [spawner-6]: process started with pid [9635]
[INFO] [spawner-7]: process started with pid [9637]
[robot_state_publisher-1] [WARN] [1773383518.815694424] [kdl_parser]: The root link base_link has an inertia specified in the URD
but KDL does not support a root link with an inertia.  As a workaround, you can add an extra dummy link to your URDF.
[robot_state_publisher-1] [INFO] [1773383518.815824354] [robot_state_publisher]: got segment Camera_Link
[robot_state_publisher-1] [INFO] [1773383518.815886691] [robot_state_publisher]: got segment Gripping_point_Link
[robot_state_publisher-1] [INFO] [1773383518.815895376] [robot_state_publisher]: got segment Llink1_Link
[robot_state_publisher-1] [INFO] [1773383518.815901628] [robot_state_publisher]: got segment Llink2_Link
[robot_state_publisher-1] [INFO] [1773383518.815907310] [robot_state_publisher]: got segment Llink3_Link
```

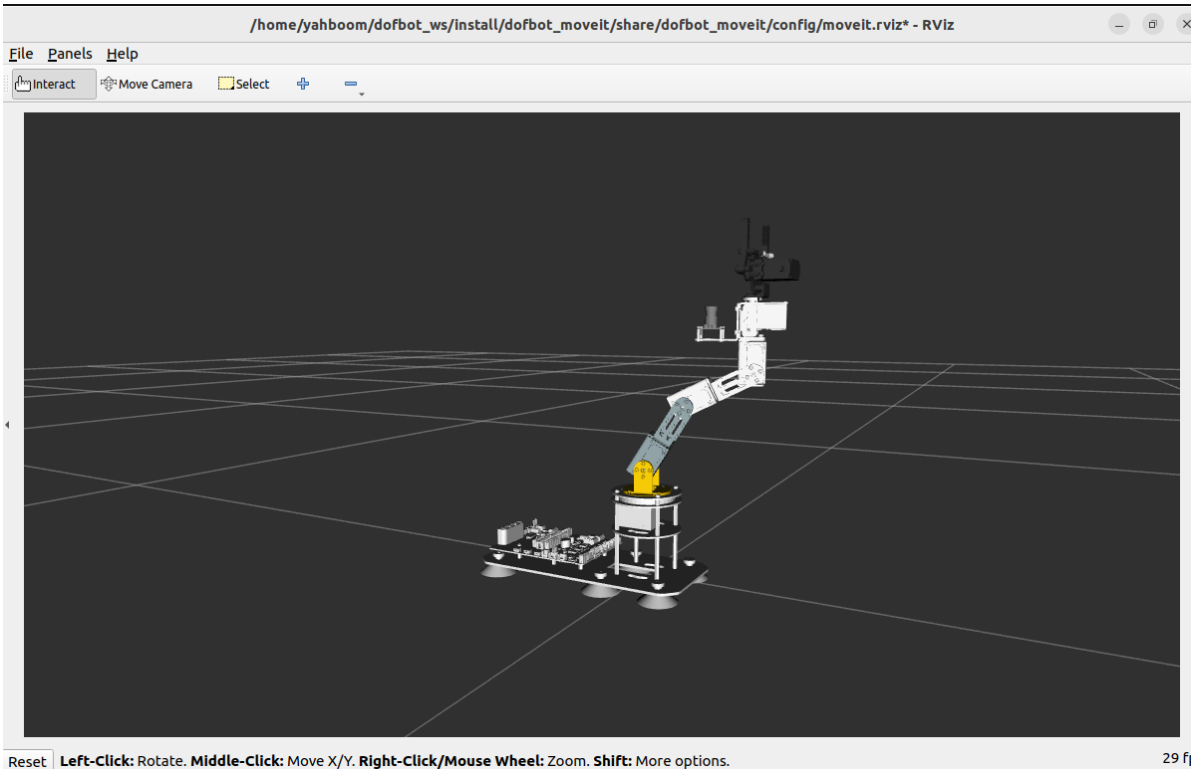
3.1 Inverse Kinematics Design

Inverse kinematics refers to the process where, given the position of the robotic arm's end effector, MoveIt calculates the angles of each joint of the robotic arm and then automatically plans the target pose.

Launch Command

Once the robotic arm has successfully loaded in MoveIt and the message `You can start planning now!` appears, execute the following command: the robotic arm will automatically plan its movement to the target position.

```
# Execute this command within the virtual machine
ros2 run dofbot_moveit set_target_position
```



3.2. Source Code Analysis

Source Code Path:

```
# Path within the virtual machine
~/dofbot_ws/src/dofbot_moveit/src/set_target_position.cpp
```

```
#include <rclcpp/rclcpp.hpp>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <geometry_msgs/msg/pose.hpp>
#include <random>

class SetTargetPosition : public rclcpp::Node
{
public:
  SetTargetPosition()
    : Node("set_target_position")
  {
   // Initialize other content
    RCLCPP_INFO(this->get_logger(), "Initializing RandomMoveIt2Control.");
  }

  void initialize()
  {
    // Initialize move_group_interface_ and planning_scene_interface_ in this
function_
    move_group_interface_ =
std::make_shared<moveit::planning_interface::MoveGroupInterface>
(shared_from_this(), "arm_group");
    planning_scene_interface_ =
std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    move_group_interface_->setNumPlanningAttempts(10);  // Set the maximum
number of planning attempts to 10
    move_group_interface_->setPlanningTime(5.0);         // Set the maximum
planning time per attempt to 5 seconds.
    // Set up the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<> dis(-0.1, 0.1);
    // Plan Path
    moveit::planning_interface::MoveGroupInterface::Plan my_plan;

    move_group_interface_->setNamedTarget("up");
    bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
    if (success)
    {
        RCLCPP_INFO(this->get_logger(), "Init arm successed.");
        move_group_interface_->execute(my_plan);
    }
    else
    {
        RCLCPP_ERROR(this->get_logger(), "Init arm failed!");
    }
```

```
      geometry_msgs::msg::PoseStamped pose_up = move_group_interface_-
>getPoseTarget();
      RCLCPP_INFO(this->get_logger(), "pose_up: position.x=%f, position.y=%f,
position.z=%f, orientation.x=%f, orientation.y=%f, orientation.z=%f,
orientation.w=%f" ,
                    pose_up.pose.position.x,
                    pose_up.pose.position.y,
                    pose_up.pose.position.z,
                    pose_up.pose.orientation.x,
                    pose_up.pose.orientation.y,
                    pose_up.pose.orientation.z,
                    pose_up.pose.orientation.w);

    // Set target position
    geometry_msgs::msg::Pose target_pose;
    target_pose.orientation.x = -0.013;
    target_pose.orientation.y = 0.192;
    target_pose.orientation.z = 0.058;
    target_pose.orientation.w = 0.707075;
    target_pose.position.x = 0.111138;
    target_pose.position.y = 0.028503;
    target_pose.position.z = 0.311743;

    // Move the robotic arm's end effector to the specified position.
    move_group_interface_->setPoseTarget(target_pose);


    bool success_position = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);

    if (success_position)
    {
      RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm.");
      move_group_interface_->execute(my_plan);
      rclcpp::sleep_for(std::chrono::seconds(5));
            // Get the current pose of the robot arm's end-effector.
      geometry_msgs::msg::PoseStamped pose_target = move_group_interface_-
>getPoseTarget();
      RCLCPP_INFO(this->get_logger(), "pose_target: position.x=%f,
position.y=%f, position.z=%f, orientation.x=%f, orientation.y=%f,
orientation.z=%f, orientation.w=%f" ,
                    pose_target.pose.position.x,
                    pose_target.pose.position.y,
                    pose_target.pose.position.z,
                    pose_target.pose.orientation.x,
                    pose_target.pose.orientation.y,
                    pose_target.pose.orientation.z,
                    pose_target.pose.orientation.w);
    }
    else
    {
      RCLCPP_ERROR(this->get_logger(), "Planning failed!");
    }

  }

private:
```

```
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
};

int main(int argc, char **argv)
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<SetTargetPosition>();

 // Lazy Initialization
  node->initialize();

  rclcpp::spin(node);
  rclcpp::shutdown();
  return 0;
}
```